PRACTICAL LAB 1

Move the Real SO-101 Robot Arm

Joint motion, live monitoring, stiffness and damping, and a gripper exercise on real hardware

Name		**Student ID**	
Group / bench		**Date**	
Arm COM port		**Calibration ID (SO101_ID)**	

Lab overview

Item	Details
Duration	About 1 hour for Steps 1–8, plus 30 minutes for the gripper exercise
Equipment	SO-ARM101 follower arm (assembled, motor IDs 1–6 set, base clamped), controller board, power supply, USB cable, Windows PC
Software	Conda env `lerobot` (LeRobot 0.6.2) at `C:\Users\cheeh\.conda\envs\lerobot`, programs in `C:\SO101wClaude\tool`
Recommended before	Isaac Sim lab "Test SO101 arm movement" (`C:\SO101wClaudeIsaac\docs\test-so101-arm-movement.md`)
Hand in	The completed result tables, the sign-off checklist, answers to the post-lab questions, and your two CSV logs / plots

Learning objectives

By the end of this lab you will be able to:

- Connect to and calibrate the SO-101 follower arm with LeRobot.
- Command a joint along a timed trajectory from Python, and stop the arm safely.
- Read the servo feedback (position, target, error, load, current, temperature) and decide whether a joint is fine, slow, weak or at a limit.
- Relate the servo's P and D gains to the simulator's stiffness and damping.
- Change a working program to create a new motion (gripper open/close).

Introduction

This lab is the real-hardware version of the Isaac Sim test `C:\SO101wClaudeIsaac\docs\test-so101-arm-movement.md`. Instead of a simulated arm, you make the **real SO-ARM101 follower arm** swing forward and backward 10 times.

You also monitor every joint while it moves (position, target, error and load), save a log, plot it, and then turn the program into a gripper open/close program in the exercise.

It takes about 1 hour, plus 30 minutes for the exercise.

Safety first

A real arm can hit you, the table or itself. Before every run:

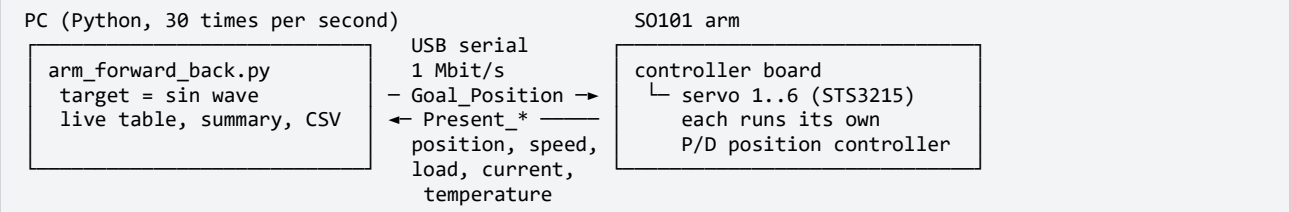
- ☐ **Clamp the arm's base** to the table edge.
- ☐ **Clear the arm's reach**: about 40 cm all around, nothing on the table in its path, no cables in its way.
- ☐ **Keep one hand near the power switch or plug.** Cutting power stops every motor at once. The arm then goes limp and falls, so be ready to catch it.
- ☐ **Press Ctrl+C** in the terminal to stop a program. The arm stops moving and holds its pose.
- ☐ **Start small and slow.** The default swing is ±20° over 3 s. Make it bigger only after a clean run.
- ☐ **Never squeeze the gripper on something for a long time.** The servo heats up. The program stops if any servo goes above 55 °C.

What you need

Item	Value
Arm	SO-ARM101 **follower** arm, assembled, motor IDs set (1–6), base clamped
Controller board	Connected to the PC by USB and to its power supply
Python	`C:\Users\cheeh\.conda\envs\lerobot\python.exe` (conda env `lerobot`, LeRobot 0.6.2)
Movement program	`tool\arm_forward_back.py`
Joint monitor	`tool\joint_monitor.py` (its table is also built into the movement program)
Joint log plotter	`tool\plot_joint_log.py`
Joint logs (CSV)	`logs\arm_forward_back_<date>_<time>.csv`
Arm library	`tool\so101_arm.py` (LeRobot `SO101Follower` plus smooth moves, soft limits, gains)
Command-line tool	`tool\so101_cli.py` (ports, calibrate, read, move, torque, gains)
Exercise solution	`tool\solutions\gripper_open_close.py`

All commands are run from `C:\SO101wClaude` in **PowerShell** and call the `lerobot` Python by its full path. You don't need to activate an environment.

How it works



- In the sim, two terminals were needed: one ran Isaac Sim and the other sent the program in. On the real arm you need **one terminal**. The program runs on the PC and talks to the servos over the USB serial port.
- Each STS3215 servo runs its own position controller (P = stiffness, D = damping). The program only sends **targets** (`Goal_Position`) and reads **feedback** (`Present_Position`, `Present_Velocity`, `Present_Load`, `Present_Current`, `Present_Temperature`).
- **Only one program can open the COM port at a time.** Close the joint monitor before you run the movement program. The movement program shows the same table itself.

Step 1: Connect the arm and find its port

1. Plug the power supply into the follower's controller board, then plug the USB cable into the PC.
2. Open PowerShell and run:

```
cd C:\SO101wClaude
$py = "C:\Users\cheeh\.conda\envs\lerobot\python.exe"
& $py -m pip install matplotlib      # once, for the plotter in Step 6d
& $py tool\so101_cli.py ports
```

Example output:

```
COM5  USB-Enhanced-SERIAL CH343 (COM5)
```

If you aren't sure which port is the arm, run `& "C:\Users\cheeh\.conda\envs\lerobot\Scripts\lerobot-find-port.exe"`. It asks you to unplug the USB cable and tells you which port disappeared.

3. Tell the tools which port and calibration name to use. These variables only last for this terminal:

```
$env:SO101_PORT = "COM5"                    # your port from above
$env:SO101_ID   = "my_awesome_follower_arm"   # any name; it selects the calibration file
```

> **Keep the `&` at the start:** `& $py ...` tells PowerShell to run the program stored in `$py`. Without it you get `Unexpected token 'tool\so101_cli.py'`.

Step 2: Calibrate the arm (once per arm)

Calibration teaches LeRobot where each joint's middle and ends are. It saves them to `%USERPROFILE%\.cache\huggingface\lerobot\calibration\robots\so_follower\<SO101_ID>.json`. Skip this step if you've already calibrated this arm under the same `SO101_ID`.

```
& $py tool\so101_cli.py calibrate
```

1. When asked, move the arm by hand to the **middle of its range**: upper arm upright, forearm pointing forward, wrist straight, gripper half open. This is the diagram LeRobot prints. Then press **ENTER**.
2. Move **every joint except wrist_roll** slowly through its **full range**, end to end. Then press **ENTER**.

The middle pose you held in step 1 becomes **0°** for every joint. The program in Step 5 swings the arm around this pose.

> **Note:** If the motors have never had their IDs set, run this first and follow its prompts: `& "C:\Users\cheeh\.conda\envs\lerobot\Scripts\lerobot-setup-motors.exe" --robot.type=so101_follower --robot.port=$env:SO101_PORT`

Step 3: Check the connection

```
& $py tool\joint_monitor.py --once
```

You should see one row per joint. Example (your numbers will differ):

```
SO-101 (torque ON - holding)
joint              pos  target   error     vel  load %     mA  temp  limits (min .. max)
 shoulder_pan      +1.2    +1.2    +0.0    +0.0    +0.0      0    31  [#######.......] -108 .. +108
 shoulder_lift    -96.4   -96.4    +0.0    +0.0    -4.2     26    32  [..............] -98 .. +98
 elbow_flex       +94.8   +94.8    +0.0    +0.0    +3.1     19    32  [##############.] -96 .. +96
 wrist_flex       +62.0   +62.0    +0.0    +0.0    +0.0      0    31  [###########...] -100 .. +100
 wrist_roll        -3.5    -3.5    +0.0    +0.0    +0.0      0    30  [#######.......] -178 .. +178
 gripper %         +1.6    +1.6    +0.0    +0.0    +0.0      0    31  [..............] +0 .. +100
```

This is the real-arm version of `print_joints()` in the sim. If it fails, see Troubleshooting.

Step 4: Look at the arm at rest, and find each joint's + direction

In the sim, + and − were fixed by the robot model. On the real arm they come from your calibration, so find them out **before** the arm moves on its own.

1. **Hold the arm with one hand.** It goes limp in the next command.
2. Run:

```
& $py tool\joint_monitor.py --torque-off
```

3. Move each joint gently by hand and watch its `pos` value. Fill in this table:

Joint	Physical direction that makes `pos` go up (+)
shoulder_pan	
shoulder_lift	
elbow_flex	
wrist_flex	
wrist_roll	
gripper	open or close?

4. Put the arm in the calibration middle pose and check that all values are close to 0. That is where Step 5 moves the arm before it swings.
5. Put the arm back in its rest pose, press **Ctrl+C**, then switch the torque back on. **The arm doesn't jump**: the tool sets every servo's target to where it is now before it turns torque on.

```
& $py tool\so101_cli.py torque on
```

Step 5: Run the arm movement test

```
& $py tool\arm_forward_back.py
```

1. The program first shows where every joint will go and waits:

```
[arm_forward_back] planned move to START_POSE (3 s):
    shoulder_pan      +1.2 ->    +0.0
    shoulder_lift    -96.4 ->    +0.0
    elbow_flex       +94.8 ->    +0.0
    ...
[arm_forward_back] then shoulder_lift moves between -20.0 and +20.0 deg, 10 x 3.0s
Clear the area around the arm, keep a hand near the power switch. ENTER = start, Ctrl+C = cancel
```

   Check that the move makes sense, then press **ENTER**.

2. The arm moves to the middle pose in 3 s, then swings `shoulder_lift` ±20° ten times, about 30 s in total. A live joint table refreshes in the terminal, with the swung joint marked `>`.
3. After each swing, a summary line appears above the table. At the end you get a summary of every joint and the path of the CSV log. Then the arm moves back to the pose it started in and holds it.

The output looks like this. These numbers come from a dry run with a simulated servo, **not a real arm**. Yours will differ, and recording them is part of the lab.

```
Started: 10 x 3.0s swings of shoulder_lift (+/-20.0 deg), ~30s total. Ctrl+C = stop
[arm_forward_back] cycle 1/10  shoulder_lift reached -18.6 .. +18.7 deg from start (commanded +/-20.0)  max
error 2.6 deg  load saturated 0% of steps
...
[arm_forward_back] done: 10 cycles, average 30 samples/s (asked for 30)
[arm_forward_back] joint summary over 300 steps:
  joint            min pos  max pos  max |err|  max |load|%  sat %  max C
  shoulder_pan        +0.0     +0.0        0.0          0.0      0     33
  shoulder_lift      -18.6    +18.7        2.6         15.6      0     33
  ...
[arm_forward_back] joint log saved: C:\SO101wClaude\logs\arm_forward_back_<date>_<time>.csv
[arm_forward_back] returning to the pose the arm started in
```

You can run Step 5 as often as you like. Each run writes a new CSV. Add `--yes` to skip the ENTER prompt once you trust the motion.

Step 6: Monitor the joints

Each joint has the same four key values as in the sim, plus three values the sim doesn't have:

Column	Meaning	Unit	Sim equivalent
pos	Where the joint actually is (`Present_Position`)	deg (gripper %)	position (rad)
target	Where the servo is told to go (`Goal_Position`)	deg (gripper %)	target (rad)
error	target − pos. **Yellow** above 3°.	deg	error
vel	Speed (`Present_Velocity`)	deg/s	velocity
load %	How hard the motor pushes, in % of its maximum torque (`Present_Load`). **Red** `SAT` at 95 % of the servo's `Torque_Limit`.	%	torque (N·m)
mA	Motor current (`Present_Current`, about 6.5 mA per unit)	mA	–
temp	Servo temperature (read once per second)	°C	–
limits	Where the joint sits inside its **calibrated** range. **Red** within 3 % of either end.	bar	position within limits

Unit conversions, so you can compare with the sim: 1 rad = 57.3°, and the sim's ±0.6 rad swing is ±34°. The STS3215 stalls at about 1.9 N·m, so 100 % load ≈ 1.9 N·m and the sim's 1.9 N·m `SAT` corresponds to about 100 % here. The **gripper is limited to 50 %** by LeRobot to protect the motor, so its `SAT` shows at about 48 %.

6a. Live table (during the run)

This is built into `arm_forward_back.py` (`SHOW_MONITOR = True`). It's the real-arm version of the sim's SO101 Joint Monitor window.

6b. Snapshot or live view (arm not moving)

```
& $py tool\joint_monitor.py --once      # one snapshot
& $py tool\joint_monitor.py             # live, Ctrl+C to stop (torque stays on)
```

6c. Per-swing summary

- **reached A .. B deg from start**: the furthest the joint actually got back (A) and forward (B), compared with the commanded `+/-AMPLITUDE`.
- **max error**: the largest gap between target and position in that swing.
- **load saturated N% of steps**: how often the servo was at its torque limit.
- **clipped N% of steps** (only shown if non-zero): how often the target was outside the calibrated range and was cut back. The sim had no such protection.

In the joint summary, joints that weren't commanded to move should barely change. If they do, the swing is shaking the rest of the arm.

6d. CSV log and plot

Every sample is saved with columns `step`, `t`, then `pos_`, `target_`, `vel_`, `load_`, `current_` and `temp_` for all six joints. To plot it (the arm doesn't need to be connected):

```
& $py tool\plot_joint_log.py                    # newest log, opens a window
& $py tool\plot_joint_log.py --out joints.png   # save as an image instead
```

You get one row per joint: position vs target (left) and load (right). The gap between the green line (position) and the grey line (target) is the tracking error.

Record your results

Question	Your run (shoulder_lift ±20°, 3 s)	Sim (shoulder_lift ±0.6 rad, 2 s)
Commanded range	−20 .. +20°	−34 .. +34°
Reached range		−15 .. +28°
Largest error		33° (0.58 rad)
Load saturated		97 % of steps
Hottest servo		–
Verdict: fine / slow / weak / at a limit		not weak, damping 65 too high

How to read your numbers:

What you see	What it means
Small error, low load	The joint follows easily.
Large error, low load	**Slow**: the servo isn't trying hard. Its stiffness (P) is low, or the motion is too fast.
Large error, load at `SAT`	**Weak or blocked**: the motor is pushing as hard as it can. The load is too heavy, or the joint is pressing against something.
`clipped` > 0 %	**At a limit**: you asked for more than the calibrated range allows.
Lopsided reached range (e.g. −15 .. +19)	Gravity helps the joint one way and resists it the other way.

Step 7: Stiffness and damping on the real arm

The sim's PD drive (`stiffness`, `damping`) is the real servo's `P_Coefficient` and `D_Coefficient`. They use the servo's own units, so the numbers can't be compared with the sim directly. LeRobot sets **P = 16, D = 32, I = 0** on every joint each time it connects.

Show the current values:

```
& $py tool\so101_cli.py gains
```

Run each experiment below:

1. Change **one** value with the command shown.
2. Run Step 5 again.
3. Compare the summary and plot with the default run.

Without `--save`, each change lasts only until the next connect. Because each program connects anew, the test run would then use the defaults again. So use `--save`, and **put the default back** afterwards.

Experiment	Command	What to look for
Softer shoulder	`& $py tool\so101_cli.py gains shoulder_lift --p 8 --save`	Bigger error, more sag under gravity
Stiffer shoulder	`& $py tool\so101_cli.py gains shoulder_lift --p 32 --save`	Smaller error. Listen for buzzing (LeRobot lowered the factory 32 to 16 to avoid shaking)
Less damping	`& $py tool\so101_cli.py gains shoulder_lift --d 8 --save`	Overshoot at the turn-around points
Lower torque limit	`& $py tool\so101_cli.py gains shoulder_lift --torque_limit 400 --save`	`SAT` appears at 38 % load; the joint can't follow
Back to default	`& $py tool\so101_cli.py gains shoulder_lift --p 16 --d 32 --torque_limit 1000 --save`	

Saved values are kept in `tool\gains.json` and re-applied every time a tool connects. Delete the file to go back to LeRobot's defaults everywhere.

> **Note:** Matching the sim to the real arm works the other way round: record a real log here, then tune the sim's `stiffness`/`damping` until its log looks the same.

Step 8: Stop the test

- **Normal end:** the program returns the arm to its start pose and exits. Torque stays **on**, so the arm holds its pose.
- **Ctrl+C:** the motion stops immediately and the arm holds where it is. The log is still saved.
- **To let the arm go limp:** support it, then run `& $py tool\so101_cli.py torque off`.
- **Emergency:** switch off or unplug the arm's power supply, and catch the arm.

Change the movement

Edit section 1 at the top of `tool\arm_forward_back.py`, then run Step 5 again:

Setting	Default	Meaning
`CYCLES`	`10`	Number of forward-and-back swings
`PERIOD_S`	`3.0`	Seconds for one swing
`AMPLITUDE`	`20.0`	How far the joint moves from `START_POSE`, in degrees (gripper: %)
`JOINT`	`"shoulder_lift"`	`shoulder_pan`, `shoulder_lift`, `elbow_flex`, `wrist_flex`, `wrist_roll` or `gripper`
`START_POSE`	all `0.0`, gripper `None`	Centre of the swing. `None` = leave that joint where it is
`MOVE_TIME_S`	`3.0`	Time to move to `START_POSE` and back
`RETURN_TO_START`	`True`	End in the pose the arm started in

Monitoring and safety settings (section 2):

Setting	Default	Meaning
`RATE_HZ`	`30`	Commands and feedback samples per second
`SHOW_MONITOR`	`True`	Live joint table. `False` = summaries and CSV only
`SAVE_CSV`	`True`	Save the joint log to `logs\`
`SATURATION_FRACTION`	`0.95`	Load at or above this fraction of `Torque_Limit` counts as saturated
`MAX_TEMP_C`	`55`	Stop if any servo gets hotter than this

For example, `JOINT = "shoulder_pan"` swings the arm left and right. Go up to bigger swings in steps: 20° → 30° → 40°. Keep an eye on `load %` and `clipped`.

The Python code

File	What it does
`tool\arm_forward_back.py`	The movement program: settings, move to start, swing loop, summaries, CSV. Full listing in the appendix
`tool\joint_monitor.py`	Reads the servo feedback registers (`read_joints`), draws the live table (`render_table`, `LiveDisplay`)
`tool\so101_arm.py`	`SO101Arm`: connect, `get_positions`, `send` (clipped to the calibrated range), `move_to` (smooth), `torque_on/off`, `set_gains`
`tool\plot_joint_log.py`	Plots a CSV log
`tool\so101_cli.py`	Command line: `ports`, `calibrate`, `read`, `move`, `joint`, `gripper`, `home`, `torque`, `pose`, `gains`

The heart of the movement program is short. Every 1/30 s it computes the target, sends it, and reads back every joint:

```
def target_at(t: float, centre: float) -> float:
    """Where JOINT should be t seconds after the swing started."""
    # Sine wave around the start pose: 0 -> +A -> 0 -> -A -> 0 once every PERIOD_S seconds.
    return centre + AMPLITUDE * math.sin(2 * math.pi * t / PERIOD_S)


...
while (t := time.perf_counter() - t0) < CYCLES * PERIOD_S:
    wanted = target_at(t, centre)
    targets[JOINT] = arm.send({JOINT: wanted}, warn=False)[JOINT]   # clipped to the calibrated range
    sample = read_joints(arm, with_temperature=read_temp)          # pos, vel, load, current, temp
    rows.append(log_row(step, t, sample, targets))                 # CSV row
    ...
    step += 1
    precise_sleep(max(0.0, t0 + step / RATE_HZ - time.perf_counter()))
```

The same idea in a few lines of your own Python, using the arm library:

```
import sys; sys.path.insert(0, r"C:\SO101wClaude\tool")
from so101_arm import SO101Arm

with SO101Arm(port="COM5", robot_id="my_awesome_follower_arm") as arm:
    arm.move_to({"shoulder_pan": 0, "shoulder_lift": 0, "elbow_flex": 0, "wrist_flex": 0, "wrist_roll": 0},
duration=3)
    for _ in range(3):
        arm.move_joint("shoulder_lift", 20, duration=1.5)
        arm.move_joint("shoulder_lift", -20, duration=1.5)
    arm.move_joint("shoulder_lift", 0, duration=1.5)
```

Troubleshooting

Problem	Cause and fix
`No serial ports found`	The USB cable isn't plugged in, or the driver for the board's USB chip (CH340/CH343) is missing. Check Device Manager → Ports (COM & LPT).
`could not open port 'COM5': PermissionError` / `Access is denied`	Another program has the port open: the joint monitor, another terminal, or a serial monitor. Close it.
`Failed to sync read 'Present_Position' ...`	The arm's power supply is off or disconnected, a 3-pin cable is loose, or the wrong port was used. With a Waveshare board, set both jumpers to **B** (USB).
`FeetechMotorsBus motor check failed on port ...` / `Missing motor IDs`	Motor IDs aren't set. Run `lerobot-setup-motors` (Step 2 note).
The program asks to calibrate	No calibration file for this `SO101_ID`, or it doesn't match the motors. Run Step 2, or set `SO101_ID` to the name you used before.
`The expression after '&' in a pipeline element produced an object that was not valid`	`$py` isn't set in this terminal. Run the `$py = ...` line from Step 1 again. Variables don't carry over between terminals.
The arm moves the wrong way	Directions come from calibration. Use your table from Step 4, and flip the sign of `AMPLITUDE` if needed.
`WARNING: ... targets will be clipped`	The swing goes past the calibrated range. Make `AMPLITUDE` smaller or move `START_POSE`.
`stopped: <joint> is at 56 C`	A servo is too hot. Let the arm rest with torque off for 10 minutes. Make the motion lighter (smaller amplitude, less load).
Joint buzzes or shakes while holding	P too high or D too high. Reset gains to default (Step 7).
`average 18 samples/s (asked for 30)`	The USB serial link is slow. Lower `RATE_HZ`, or set the COM port's latency timer to 1 ms (Device Manager → Port → Advanced).
`ModuleNotFoundError: matplotlib`	Run `& $py -m pip install matplotlib`.
The live table prints garbage like `←[31m`	Very old console. Use Windows Terminal or the VS Code terminal, or set `SHOW_MONITOR = False`.

Exercise: Open and close the gripper

In this exercise you turn the arm-swing program into a gripper program, run it on the real arm, and monitor the gripper the same way you monitored the shoulder. It takes about 30 minutes.

You will practise:

- Copying a working program and changing only what you need.
- Writing a motion for a joint that only moves one way.
- Reading the monitor to decide whether a joint is **weak**, **slow**, or **at a limit**.

What you need

Item	Path
Starting program (you copy it)	`tool\arm_forward_back.py`
Your new program (you create it)	`tool\gripper_open_close.py`
Log plotter	`tool\plot_joint_log.py`
Reference solution	`tool\solutions\gripper_open_close.py`

Facts about the real gripper:

Property	Value
Joint name	`gripper` (servo ID 6)
Unit	% of its calibrated range: 0 = one end, 100 = the other. On most calibrations, **0 ≈ closed**; you check this in E1
Torque limit	50 % of the servo's maximum, set by LeRobot so it doesn't burn out
Protection	`Protection_Current` 50 %; `Overload_Torque` 25 % when overloaded too long
Servo	STS3215, stalls at about 1.9 N·m; continuous rating much lower

Why the arm program can't be reused as-is

`arm_forward_back.py` moves the joint along a sine wave **around** the start pose: 0 → +A → 0 → −A → 0. That suits the shoulder, which can move both ways. The gripper starts closed, so the −A half would push the jaw into its closed stop.

The gripper needs a wave that goes **only one way**, closed → open → closed:

```
target = closed + A × 0.5 × (1 − cos(2π t / T))
```

Time in the cycle	0	T/4	T/2	3T/4	T
`0.5 × (1 − cos(...))`	0	0.5	1	0.5	0
Gripper	closed	half open	fully open (A)	half open	closed

The wave also starts and ends with zero speed, so the jaw doesn't jerk.

Step E1: Look at the gripper at rest

1. Hold the arm, then run `& $py tool\joint_monitor.py --torque-off`.
2. Close the jaw fully by hand and note the `gripper %` value. Open it fully and note it again.
   - If **closed ≈ 0** (the usual case), use `2.0` as "closed" and a positive `AMPLITUDE`.

- If **closed ≈ 100**, use `98.0` as "closed" and a **negative** `AMPLITUDE` (`-60.0`).

3. Ctrl+C, then `& $py tool\so101_cli.py torque on`.

Step E2: Copy the program

```
Copy-Item tool\arm_forward_back.py tool\gripper_open_close.py
code tool\gripper_open_close.py
```

Make all the changes below in **`tool\gripper_open_close.py`**, not in the original.

Step E3: Make the five changes

Change 1: settings. In section 1, change these four lines:

```
# before
CYCLES = 10                # number of forward-and-back swings
PERIOD_S = 3.0             # seconds for one swing
AMPLITUDE = 20.0           # how far JOINT moves from START_POSE: degrees (gripper: %)
JOINT = "shoulder_lift"

# after
CYCLES = 5                 # 5 open/close cycles
PERIOD_S = 3.0             # 3 s per cycle
AMPLITUDE = 60.0           # open by 60 % of the gripper's calibrated range
JOINT = "gripper"
```

In `START_POSE`, change `"gripper": None,` to the closed value from E1:

```
    "gripper": 2.0,        # closed (check which end is closed in Step E1)
```

Change 2: the program's name. Press **Ctrl+H**, find `arm_forward_back`, replace with `gripper_open_close`, and click **Replace All**. It should make **3 replacements**. This changes the labels in the output (`[gripper_open_close] ...`) and the log file name (`logs\gripper_open_close_<time>.csv`).

Change 3: the motion. In section 7, inside `def target_at(...)`, replace:

```
    return centre + AMPLITUDE * math.sin(2 * math.pi * t / PERIOD_S)
```

with:

```
    return centre + AMPLITUDE * 0.5 * (1 - math.cos(2 * math.pi * t / PERIOD_S))
```

Change 4: the per-cycle summary text. In `print_cycle_summary`, replace `(commanded +/-{AMPLITUDE})` with `(commanded 0 .. +{AMPLITUDE})`.

Change 5: the start message. In `main`, in the `display.log(f"Started: ...` line, replace `swings of {JOINT} (+/-{AMPLITUDE} {UNIT})` with `open/close cycles of {JOINT} (0 .. +{AMPLITUDE} {UNIT})`.

Save (**Ctrl+S**), then check your work:

```
& $py -m py_compile tool\gripper_open_close.py
code --diff tool\gripper_open_close.py tool\solutions\gripper_open_close.py
```

The first command prints nothing if there's no syntax error. In the diff, the solution only adds an updated description at the top and some rewritten comments. Every line of code should match.

Step E4: Run it

```
& $py tool\gripper_open_close.py
```

Check the planned move. It should say `then gripper moves between +2.0 and +62.0 %` with **no** clipping warning. Then press **ENTER**. The arm moves to the middle pose with the jaw closed. The jaw then opens and closes 5 times in about 15 s, and the arm goes back to its start pose.

Step E5: Monitor the gripper

1. **Live table:** watch the `gripper %` row. Does the error turn yellow? Does the load show `SAT`?
2. **Per-cycle summary:** one line per cycle, then the joint summary.
3. **Plot:** `& $py tool\plot_joint_log.py` and look at the bottom row.

Step E6: Read your results

Question	Gripper (this exercise)	Shoulder (Step 5)
Commanded range	2 .. 62 %	−20 .. +20°
Reached range		
Largest error		
Load saturated		
Largest speed (`vel`)		
Verdict: fine / slow / weak / at a limit		

Compare with the sim's gripper, which was **slow, not weak**: its damping-to-stiffness ratio gave a delay of 0.33 s and it never reached `SAT`. Is your real gripper slow? Weak? Neither?

Step E7: Experiments

Write down your **prediction** first, then change one setting in `tool\gripper_open_close.py`, run E4 again and compare. Put the setting back before the next experiment.

Experiment A: 3× faster. Set `PERIOD_S = 1.0`. Does the jaw still open to 62 % and close to 2 %? Which column shows that it can't keep up: error, vel or load?

Experiment B: try to open past the limit. Set `AMPLITUDE = 150.0`.

- The program warns before it starts, and the summary shows `clipped N% of steps`. `SO101Arm` cuts every target to the calibrated range (0–100 %). Unlike the sim, the target never goes past the end, so the jaw doesn't press into its stop.
- Now hold a soft object (foam, a sponge) in the jaw with `AMPLITUDE = 60.0`, closing on it. Watch `load %` reach `SAT` near 48 % while the jaw is blocked. This is the same situation as the sim's Experiment B. Keep it short, and watch `temp`.

Experiment C: does the gripper disturb the arm? Keep the original settings and look at the other joints' rows in the joint summary and the plot. The wrist carries the gripper motor: does its load ripple in time with the gripper?

What this means for the sim

- **Units:** the real gripper reports % of its range. The sim reports radians. Map them with your E1 readings (closed/open %) and the sim's range (0 .. 1.65 rad).
- **Grip strength:** the sim allows 2.5 N·m. The real gripper is capped at about 50 % of 1.9 N·m.
- **Delay:** compare your real plot's lag with the sim's 0.33 s. If they differ, the sim's stiffness/damping for the gripper can be tuned to match.

Exercise troubleshooting

Problem	Cause and fix
`SyntaxError`, or `py_compile` reports an error	A typo in one of the changes. Compare with the solution using `code --diff`.
`ValueError: JOINT = 'grippr' is not one of ...`	`JOINT` is misspelt. It must be exactly `"gripper"`.
Output still says `[arm_forward_back]`, or the CSV is named `arm_forward_back_...`	Change 2 wasn't done or wasn't saved.
`WARNING: ... targets will be clipped` with `AMPLITUDE = 60`	Change 3 is missing (still `math.sin`), or "closed" in `START_POSE` is at the wrong end (E1).
The jaw squeezes shut at the start	`START_POSE["gripper"]` is set to the wrong end. Redo E1.

Lab sign-off

Tick each task as you complete it. Your instructor initials the last column.

#	Task	Done	Instructor
1	Port found and arm calibrated (Steps 1–2)	☐	
2	Connection checked; + direction table filled in (Steps 3–4)	☐	
3	Swing test run; CSV log saved and plotted (Steps 5–6)	☐	
4	“Record your results” table completed	☐	
5	Stiffness/damping experiments done, and gains put back to default (Step 7)	☐	
6	Gripper exercise run; E6 table completed	☐	
7	Arm left in its rest pose, torque off with the arm supported, power off	☐	

Post-lab questions

Answer in the space provided. Use the numbers from your own result tables.

Q1. Explain the difference between target and position. What does a large error combined with a low load tell you about a joint?

Q2. Why does a sine wave suit shoulder_lift but not the gripper? Sketch both target curves over one period.

Q3. Was your swing’s reached range symmetric? If not, explain why, with reference to gravity.

Q4. Describe what changed when you lowered and raised P on shoulder_lift. How does this compare with the simulator's stiffness and damping?

Q5. SO101Arm clips every target to the calibrated range. What could happen on the real arm without this clipping? How did the simulator behave instead?

Appendix: arm_forward_back.py

The full movement program, as in `tool\arm_forward_back.py`:

```
"""arm_forward_back.py - swing one joint of the REAL SO-101 arm forward and back.

Real-arm version of the Isaac Sim test (SO101wClaudeIsaac\\docs\\test-so101-arm-movement.md).
The program:
  1. moves the arm smoothly to START_POSE (after you press ENTER),
  2. swings JOINT +/-AMPLITUDE around it, CYCLES times,
  3. shows a live joint table and prints a summary after every swing,
  4. prints a summary of every joint, saves a CSV log in logs\\,
  5. moves the arm back to the pose it started in.

Run from C:\\SO101wClaude:
    & $py tool\\arm_forward_back.py
Ctrl+C stops the motion at once; the arm holds where it is.
"""

import argparse
import csv
import math
import sys
import time
from datetime import datetime
from pathlib import Path

TOOL_DIR = next(p for p in Path(__file__).resolve().parents if p.name == "tool")
sys.path.insert(0, str(TOOL_DIR))

from lerobot.utils.robot_utils import precise_sleep  # noqa: E402

from joint_monitor import LiveDisplay, read_joints, read_load_limits, render_table  # noqa: E402
from so101_arm import DEFAULT_ID, DEFAULT_PORT, GRIPPER, JOINTS, SO101Arm  # noqa: E402

# ---------------------------------------------------------------------------- 1. settings
CYCLES = 10                # number of forward-and-back swings
PERIOD_S = 3.0             # seconds for one swing
AMPLITUDE = 20.0           # how far JOINT moves from START_POSE: degrees (gripper: %)
JOINT = "shoulder_lift"    # shoulder_pan, shoulder_lift, elbow_flex, wrist_flex, wrist_roll or gripper

# Pose the swing is centred on. Degrees; 0 = middle of the calibrated range (the pose you held the
# arm in during calibration). None = leave that joint where it is when the program starts.
START_POSE = {
    "shoulder_pan": 0.0,
    "shoulder_lift": 0.0,
    "elbow_flex": 0.0,
    "wrist_flex": 0.0,
    "wrist_roll": 0.0,
    "gripper": None,
}
MOVE_TIME_S = 3.0          # time to move to START_POSE, and back at the end
RETURN_TO_START = True     # finish in the pose the arm started in (e.g. folded rest)

# ---------------------------------------------------------------------------- 2. monitoring and safety
RATE_HZ = 30               # commands and feedback samples per second
SHOW_MONITOR = True        # live joint table in the terminal. False = summaries and CSV only
SAVE_CSV = True            # save the joint log to logs\ when the run ends
SATURATION_FRACTION = 0.95 # |load| at or above this fraction of the servo's Torque_Limit counts as
saturated
MAX_TEMP_C = 55            # stop if any servo gets hotter than this (the STS3215 cuts out near 70 C)

PROGRAM = "arm_forward_back"
LOG_DIR = TOOL_DIR.parent / "logs"
UNIT = "%" if JOINT == GRIPPER else "deg"



# ---------------------------------------------------------------------------- 3. connect
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__,
formatter_class=argparse.RawDescriptionHelpFormatter)
```

```
    parser.add_argument("--port", default=DEFAULT_PORT, help=f"serial port (default {DEFAULT_PORT})")
    parser.add_argument("--id", default=DEFAULT_ID, help=f"calibration id (default {DEFAULT_ID})")
    parser.add_argument("--yes", action="store_true", help="don't ask before moving")
    return parser.parse_args()



# ---------------------------------------------------------------------------- 4. move to the start pose
def go_to_start(arm: SO101Arm, ask: bool) -> tuple[dict[str, float], dict[str, float]]:
    """Show the planned move, wait for ENTER, move. Returns (pose at program start, swing centre pose)."""
    initial = arm.get_positions()
    start = {j: initial[j] if START_POSE.get(j) is None else START_POSE[j] for j in JOINTS}
    start = arm.clamp(start)
    lo, hi = arm.joint_limits[JOINT]

    print(f"[{PROGRAM}] planned move to START_POSE ({MOVE_TIME_S:g} s):")
    for j in JOINTS:
        print(f"    {j:<14} {initial[j]:+7.1f} -> {start[j]:+7.1f}")
    # Sample one period of the motion to find the range JOINT will be asked to cover.
    path = [target_at(PERIOD_S * k / 100, start[JOINT]) for k in range(101)]
    print(f"[{PROGRAM}] then {JOINT} moves between {min(path):+.1f} and {max(path):+.1f} {UNIT}, "
          f"{CYCLES} x {PERIOD_S}s")
    if min(path) < lo or max(path) > hi:
        print(f"[{PROGRAM}] WARNING: that leaves {JOINT}'s range {lo:+.1f} .. {hi:+.1f}; targets will be
clipped")
    if ask:
        input("Clear the area around the arm, keep a hand near the power switch. ENTER = start, Ctrl+C =
cancel ")

    arm.move_to(start, duration=MOVE_TIME_S)
    return initial, start



# ---------------------------------------------------------------------------- 5. per-swing summary
def new_cycle_stats() -> dict:
    return {"lo": math.inf, "hi": -math.inf, "err": 0.0, "sat": 0, "clip": 0, "n": 0}



def print_cycle_summary(display: LiveDisplay, cycle: int, s: dict, centre: float) -> None:
    n = max(1, s["n"])
    clipped = f"  clipped {100 * s['clip'] / n:.0f}% of steps" if s["clip"] else ""
    display.log(
        f"[{PROGRAM}] cycle {cycle}/{CYCLES}  {JOINT} reached {s['lo'] - centre:+.1f} .. "
        f"{s['hi'] - centre:+.1f} {UNIT} from start (commanded +/-{AMPLITUDE})  "
        f"max error {s['err']:.1f} {UNIT}  load saturated {100 * s['sat'] / n:.0f}% of steps{clipped}"
    )



# ---------------------------------------------------------------------------- 6. joint log
def log_row(step: int, t: float, sample: dict, targets: dict) -> dict:
    row = {"step": step, "t": round(t, 4)}
    for j in JOINTS:
        row[f"pos_{j}"] = round(sample[j]["pos"], 3)
        row[f"target_{j}"] = round(targets[j], 3)
        row[f"vel_{j}"] = round(sample[j]["vel"], 2)
        row[f"load_{j}"] = round(sample[j]["load"], 1)
        row[f"current_{j}"] = round(sample[j]["current"], 1)
        row[f"temp_{j}"] = sample[j]["temp"]
    return row



def save_csv(rows: list[dict]) -> Path:
    LOG_DIR.mkdir(parents=True, exist_ok=True)
    path = LOG_DIR / f"{PROGRAM}_{datetime.now():%Y%m%d_%H%M%S}.csv"
    with path.open("w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)
    return path



def print_joint_summary(rows: list[dict], load_limits: dict[str, float]) -> None:
    print(f"[{PROGRAM}] joint summary over {len(rows)} steps:")
```

```
    print(f"  {'joint':<15}{'min pos':>9}{'max pos':>9}{'max |err|':>11}{'max
|load|%':>13}{'sat %':>7}{'max C':>7}")
    for j in JOINTS:
        pos = [r[f"pos_{j}"] for r in rows]
        err = max(abs(r[f"target_{j}"] - r[f"pos_{j}"]) for r in rows)
        load = [abs(r[f"load_{j}"]) for r in rows]
        sat = sum(v >= SATURATION_FRACTION * load_limits[j] for v in load)
        temps = [r[f"temp_{j}"] for r in rows if not math.isnan(r[f"temp_{j}"])]
        print(f"  {j:<15}{min(pos):+9.1f}{max(pos):+9.1f}{err:11.1f}{max(load):13.1f}"
              f"{100 * sat / len(rows):7.0f}{max(temps, default=math.nan):7.0f}")



# ---------------------------------------------------------------------------- 7. the motion
def target_at(t: float, centre: float) -> float:
    """Where JOINT should be t seconds after the swing started."""
    # Sine wave around the start pose: 0 -> +A -> 0 -> -A -> 0 once every PERIOD_S seconds.
    return centre + AMPLITUDE * math.sin(2 * math.pi * t / PERIOD_S)



# ---------------------------------------------------------------------------- 8. main loop
def main() -> None:
    args = parse_args()
    if JOINT not in JOINTS:
        raise ValueError(f"JOINT = {JOINT!r} is not one of {JOINTS}")

    rows: list[dict] = []
    with SO101Arm(port=args.port, robot_id=args.id) as arm:
        limits, load_limits = arm.joint_limits, read_load_limits(arm)
        try:
            initial, targets = go_to_start(arm, ask=not args.yes)
        except KeyboardInterrupt:
            print(f"\n[{PROGRAM}] cancelled - arm holds where it is")
            return
        centre = targets[JOINT]

        display = LiveDisplay(SHOW_MONITOR)
        display.log(f"Started: {CYCLES} x {PERIOD_S}s swings of {JOINT} (+/-{AMPLITUDE} {UNIT}), "
                    f"~{CYCLES * PERIOD_S:.0f}s total. Ctrl+C = stop")
        stats, cycle, step = new_cycle_stats(), 0, 0
        temps: dict[str, float] = {}
        next_temp_t, next_draw_t = 0.0, 0.0
        finished = False
        t0 = time.perf_counter()
        try:
            while (t := time.perf_counter() - t0) < CYCLES * PERIOD_S:
                # Command the moving joint (clipped to its calibrated range), then read every joint back.
                wanted = target_at(t, centre)
                targets[JOINT] = arm.send({JOINT: wanted}, warn=False)[JOINT]
                read_temp = t >= next_temp_t
                sample = read_joints(arm, with_temperature=read_temp)
                if read_temp:
                    next_temp_t += 1.0
                    temps = {j: sample[j]["temp"] for j in JOINTS}
                    hottest = max(temps, key=temps.get)
                    if temps[hottest] > MAX_TEMP_C:
                        raise RuntimeError(f"{hottest} is at {temps[hottest]:.0f} C (> {MAX_TEMP_C} C)")
                for j in JOINTS:
                    sample[j]["temp"] = temps.get(j, math.nan)
                rows.append(log_row(step, t, sample, targets))

                if int(t // PERIOD_S) > cycle:
                    cycle += 1
                    print_cycle_summary(display, cycle, stats, centre)
                    stats = new_cycle_stats()
                pos = sample[JOINT]["pos"]
                stats["lo"], stats["hi"] = min(stats["lo"], pos), max(stats["hi"], pos)
                stats["err"] = max(stats["err"], abs(targets[JOINT] - pos))
                stats["sat"] += abs(sample[JOINT]["load"]) >= SATURATION_FRACTION * load_limits[JOINT]
                stats["clip"] += not math.isclose(targets[JOINT], wanted, abs_tol=1e-6)
                stats["n"] += 1

                if t >= next_draw_t:
                    next_draw_t += 0.2
```

```
                    header = f"SO-101  {PROGRAM} running  t={t:5.1f}/{CYCLES * PERIOD_S:.0f}s  cycle {cycle
+ 1}/{CYCLES}"
                    display.draw(render_table(sample, targets, limits, load_limits, header,
highlight=JOINT))

                step += 1
                precise_sleep(max(0.0, t0 + step / RATE_HZ - time.perf_counter()))
            finished = True
            print_cycle_summary(display, CYCLES, stats, centre)
            display.log(f"[{PROGRAM}] done: {CYCLES} cycles, average {len(rows) / (time.perf_counter() -
t0):.0f} "
                        f"samples/s (asked for {RATE_HZ})")
        except KeyboardInterrupt:
            display.log(f"\n[{PROGRAM}] stopped by Ctrl+C after {cycle} cycles - arm holds its current
pose")
        except Exception as e:
            display.log(f"\n[{PROGRAM}] stopped: {e} - arm holds its current pose")
        finally:
            if rows:
                print_joint_summary(rows, load_limits)
                if SAVE_CSV:
                    print(f"[{PROGRAM}] joint log saved: {save_csv(rows)}")

        if finished:
            arm.move_to({JOINT: centre}, duration=0.5)
            if RETURN_TO_START:
                print(f"[{PROGRAM}] returning to the pose the arm started in")
                arm.move_to(initial, duration=MOVE_TIME_S)
        # The arm keeps holding its pose after the program exits (torque stays on).


if __name__ == "__main__":
    main()
```